

Cross-Site Scripting (XSS)

How attacking a browser is different from attacking a database, and
how to stop it

JAVASCRIPT / NODE +
PYTHON / DJANGO
TRACKS

Learning Objectives

By the end of today's class, you will be able to:

- Explain what XSS is and how it's different from SQL Injection
- Tell stored XSS and reflected XSS apart, with an OAS Bay example of each
- Demonstrate a real XSS attack on a deliberately vulnerable notes page
- Fix that page by escaping output correctly, in JavaScript/Node.js and Python/Django
- Add two extra layers of defence: a Content Security Policy header and the HttpOnly cookie flag

1. What Is XSS?

Quick Recap: Where Does SQL Injection Attack?

On Day 2, the attacker's target was the database. The malicious input travelled from a form, into a SQL query, into the database engine. The victim was the server.

What Is XSS?

Definition: Cross-Site Scripting (XSS) happens when an attacker gets their own JavaScript to run inside someone else's browser, by hiding it in data your app displays back to other users. The victim this time is not the database, it's every other person who views that page.

Think of it like this: SQL Injection is breaking into the filing cabinet. XSS is pinning a fake, dangerous notice to the staff noticeboard, worded so that anyone who reads it gets tricked into handing over their keys. You never touched the cabinet directly, you just waited for someone else to read your note.

2. Stored vs Reflected XSS

Type	How It Works	OAS Bay Example
Stored XSS	The malicious script is saved on the server, e.g. in a database or a list, and served to every visitor who views that page	A supplier note containing a script is saved once, and fires for every technician who opens the notes page
Reflected XSS	The malicious script rides along in a link or form submission,	A malicious link like /search?term=<script>...</script> is

	and the server immediately echoes it straight back into the page, with no saving involved	emailed to a technician, and just clicking it runs the script
--	---	---

Stored is generally more dangerous, one malicious note can hit every single visitor. Reflected needs the victim to click a specific crafted link, but is often used in phishing emails for exactly that reason.

3. Why Raw HTML Is Dangerous

Your browser doesn't know the difference between HTML you wrote and HTML a user typed, it just renders whatever it's given. If a note contains a `<script>` tag and your page inserts that note as raw HTML, the browser will execute it, exactly as if you'd written it yourself.

Escaping: Turning Code Back Into Text

Escaping means converting the characters that give HTML its meaning into harmless text versions, so the browser displays them instead of acting on them:

Character	Escaped To	Why
<	<	Stops a tag from opening
>	>	Stops a tag from closing
&	&	Stops an entity reference from starting
"	"	Stops breaking out of an HTML attribute

The core rule: never insert user-typed text directly into HTML. Either escape it yourself, or use a tool that escapes it for you automatically, `textContent` in JavaScript, and Django's default template autoescaping.

4. Practical: Break, Then Fix, a Supplier Notes Page

We'll build one small, focused app: technicians can leave a note, and every note is shown on one page. No database needed today, notes just live in memory while the server is running, so we can spend our time on the actual vulnerability instead of more setup.

Step 0 - Local Setup: JavaScript / Node.js Track

```
mkdir oasbay-day3 && cd oasbay-day3  
npm init -y  
npm install express
```

JAVASCRIPT / NODE.JS · server.js

```
// server.js
const express = require('express');
const app = express();
app.use(express.urlencoded({ extended: true }));

// Notes live in memory only, reset every restart
let notes = [];

// A fake session cookie, so we have something to steal
app.use((req, res, next) => {
  res.cookie('oasbay_session', 'demo-session-abc123');
  next();
});

// >>> the GET / route from Step 1 goes here <<<

// >>> POST /notes route from Step 1 <<<

app.listen(3000, () => console.log('Running at
http://localhost:3000'));
```

Run it with `node server.js`, then open `http://localhost:3000`. Leave it running, we'll fill in the two marked routes next.

Step 0 - Local Setup: Python / Django Track

```
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
pip install django
django-admin startproject oasbay_xss .
python manage.py startapp notes
```

Add 'notes' to INSTALLED_APPS in oasbay_xss/settings.py, same as you did for accounts on Day 2.

```
PYTHON / DJANGO · urls.py
# notes/urls.py (new file)
from django.urls import path
from . import views

urlpatterns = [
    path('', views.notes_page),
    path('notes/', views.add_note),
]

# oasbay_xss/urls.py (edit existing file)
from django.urls import path, include

urlpatterns = [
    path('', include('notes.urls')),
]
```

PYTHON / DJANGO · notes/views.py

```
# notes/views.py
from django.http import HttpResponseRedirect
from django.views.decorators.csrf import csrf_exempt

# Notes live in memory only, reset every restart
notes = []

# >>> notes_page from Step 1 goes here <<<

# >>> add_note from Step 1 goes here <<<
```

Run it with `python manage.py runserver`, then open `http://localhost:8000`. Django sets its own session cookie automatically once you touch `request.session`, which is enough for today's demo.

Step 1 - The Vulnerable Notes Page (Stored XSS)

Paste this into the spots marked above in each file.

JAVASCRIPT / NODE.JS · server.js

```
// goes above app.listen() in server.js
app.get('/', (req, res) => {
  const noteList = notes.map(n => '<li>' + n + '</li>').join('');
  res.send(`
    <h2>Supplier Notes</h2>
    <form method="POST" action="/notes">
      <input name="text" placeholder="Leave a note">
      <button type="submit">Add Note</button>
    </form>
    <ul>${noteList}</ul>
  `);
});

app.post('/notes', (req, res) => {
  notes.push(req.body.text); // saved raw
  res.redirect('/');
});
```

PYTHON / DJANGO · notes/views.py

```
# goes above app.listen() equivalent
def notes_page(request):
    # forces a session cookie
    request.session.setdefault('touched', True)
    # inserted raw, no escaping
    items = ''.join(f'<li>{n}</li>' for n in notes)
    return HttpResponse(f'''
        <h2>Supplier Notes</h2>
        <form method="POST" action="/notes/">
            <input name="text" placeholder="Leave a note">
            <button type="submit">Add Note</button>
        </form>
        <ul>{items}</ul>
    ''')

@csrf_exempt
def add_note(request):
    # saved raw, no escaping
    notes.append(request.POST.get('text'))
    return HttpResponse(status=302, headers={'Location': '/'})
```

VULNERABLE Both versions build the page by gluing note text directly into an HTML string, exactly the same mistake as string-built SQL on Day 2, just one layer up, in the browser instead of the database.

OWASP: A03, *Injection (XSS)*

Step 2 - Live Attack (Do This Together in Class)

- 1. Open the notes page: <http://localhost:3000> (Node.js) or <http://localhost:8000> (Django)
- 2. In the note field, type exactly: `<script>alert('XSS: ' + document.cookie)</script>`
- 3. Submit, and watch the page pop up an alert showing the fake session cookie

- 4. Refresh the page. It fires again, every single time anyone loads it, because it's stored, not reflected

VULNERABLE Think about it: this alert box could just as easily have been a silent line of code sending document.cookie to an attacker's server. Post in the chat what you think an attacker could do with a stolen OAS Bay session cookie.

Step 3 - Fix It: Escape Before You Render

Replace the two routes with the safe versions below, in the same files.

JAVASCRIPT / NODE.JS · server.js

```
// server.js - escapes the dangerous characters
function escapeHtml(str) {
  return str
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;');
}

app.get('/', (req, res) => {
  const noteList = notes.map(n => '<li>' + escapeHtml(n) +
    '</li>').join('');
  res.send(`
    <h2>Supplier Notes</h2>
    <form method="POST" action="/notes">
      <input name="text" placeholder="Leave a note">
      <button type="submit">Add Note</button>
    </form>
    <ul>${noteList}</ul>
  `);
});
```

PYTHON / DJANGO · notes/views.py

```
# notes/views.py - templates autoescape by default
# switching to render() with a template fixes this free
from django.shortcuts import render

def notes_page(request):
    request.session.setdefault('touched', True)
    return render(request, 'notes.html', {'notes': notes})
```

PYTHON / DJANGO · notes/templates/notes.html

```
{# notes/templates/notes.html #}
<h2>Supplier Notes</h2>
<form method="POST" action="/notes/">
    <input name="text" placeholder="Leave a note">
    <button type="submit">Add Note</button>
</form>
<ul>
{% for n in notes %}
    <li>{{ n }}</li>
{% endfor %}
</ul>
```

FIX Node.js: `escapeHtml()` turns `< > & "` into their harmless text versions before they ever reach the page. Django: `{{ n }}` is autoescaped by default, this is the same protection Django gave you for free the whole time, we just weren't using a template before.

Try the same `<script>` payload again on the fixed version. You should see the literal text on the page, tags and all, instead of an alert.

Reminder: The Client-Side Version of This Rule

This same principle applies in the browser's own JavaScript, not just on the server. If you ever build a page in front-end JavaScript, never use `innerHTML` with data you didn't write yourself:

```
element.innerHTML = userText;    // DANGEROUS  
element.textContent = userText;  // SAFE
```

5. Defence in Depth: Two More Layers

Escaping output is the fix. These two are backup layers, in case something you missed still slips through.

Content Security Policy (CSP)

A CSP header tells the browser which sources of JavaScript it's allowed to run at all. Even if a script tag sneaks onto your page, a strict CSP can stop the browser from executing it.

JAVASCRIPT / NODE.JS

```
// server.js - a basic CSP header
// add near the top with your other app.use() lines
app.use((req, res, next) => {
  res.setHeader('Content-Security-Policy', "default-src 'self'");
  next();
});
```

PYTHON / DJANGO

```
# settings.py - Django 4.0+ or django-csp for older versions
SECURE_CONTENT_TYPE_NOSNIFF = True
# A full CSP setup is more involved, know what it does
# OAS Bay production settings should include one
```

HttpOnly Cookies

Setting the HttpOnly flag on a cookie means JavaScript, including an attacker's injected script, can never read it with `document.cookie`. The browser can still send it to your server normally, it just can't be read from the page.

JAVASCRIPT / NODE.JS

```
// server.js - update the cookie from Step 0  
res.cookie('oasbay_session', 'demo-session-abc123', { httpOnly:  
true });
```

PYTHON / DJANGO

```
# settings.py - session cookie is HttpOnly by default  
SESSION_COOKIE_HTTPONLY = True    # already the default
```

Go back and re-run the Step 2 attack against the HttpOnly version. The alert box will now show an empty or missing cookie, document.cookie can no longer see it, even though the XSS still technically fired.

6. Recap

- XSS attacks other users' browsers, not the database, by getting your own script to run on someone else's screen
- Stored XSS is saved and hits every visitor, reflected XSS rides along in one crafted link
- The fix is the same idea as Day 2: never insert raw user input where it can be interpreted as code, escape it or let your framework autoescape it
- textContent over innerHTML on the client, autoescaped templates over raw HTML on the server
- CSP headers and HttpOnly cookies are backup layers, not replacements for escaping your output

Assignment: Cross-Site Scripting

Due before Day 4. Work individually. Submit your before-and-after code, plus a short written explanation of what you found.

The Task

OAS Bay wants customers to be able to leave a short comment after their car service, shown on a public reviews page. Build this as a new small project (oasbay-day3-assignment), reusing the exact pattern from today's notes page.

Before you start: build a reviews page with a form (customer name, comment) and a list that displays every review submitted so far, stored in memory just like today's notes.

You Must Submit

1. A working vulnerable version: reviews are inserted into the page with no escaping, deliberately, as your starting point
2. Proof of the attack: the exact `<script>` payload you used, and a screenshot of it firing
3. A fixed version: output properly escaped (Node.js) or rendered through an autoescaping template (Django)
4. Add the `HttpOnly` flag to any cookie your app sets
5. A 3 to 5 sentence explanation, in your own words, of why the original code was vulnerable, and why stored XSS on a public reviews page is more dangerous than on a private notes page only staff can see

"If you can't trust the input, don't trust it in the output either."